DESIGNDOC — MyTerm (CS69201 Computing Lab)

Author: Mridul Kumar.

Roll – 25CS60R40.

Project: MyTerm — A Custom Terminal with X11 GUI

Subject : Computing Lab

Overview

MyTerm is a graphical shell that runs inside an X11 window. It mimics a traditional bash terminal while providing additional functionality: multiple tabs, Unicode-aware multiline input, input/output redirection, pipes, a multiWatch utility, line-editing shortcuts, job control (foreground/background), searchable history, and filename auto-completion.

This document explains how each required feature was implemented in the submitted code, the system calls and libraries used, file-layout and build/run instructions, testing notes, and known limitations.

Table of Contents

1. Graphical User Interface (X11)
2. Running external commands (fork + exec)
3. Multiline & Unicode input
4. Input/Output redirection (<, >, >>)
5. Pipe support (|) and pipelines
6. multiWatch command
7. Line navigation (Ctrl+A / Ctrl+E) and key handling
8. Interrupts and job control (Ctrl+C / Ctrl+Z, fg, bg, jobs)
9. Searchable shell history (Ctrl+R)
10. Auto-complete (Tab)
11. Multiple tabs

---

1. Graphical User Interface (X11)

Goal: Present a terminal-like interface fully inside an X11 window (no use of standard console for I/O).

Implementation highlights:

- Uses Xlib and Xlocale APIs: XOpenDisplay, XCreateSimpleWindow, XMapWindow, XNextEvent, XSelectInput, XDrawString / Xutf8DrawString, and XIC/XIM for UTF-8 input handling.
- A redraw() routine renders tabs (top bar), a scrollable text buffer, an input prompt and a cursor whose position is computed via text extents.
- Keyboard events are consumed via Xutf8LookupString() to support multi-byte UTF-8 characters.
- Mouse support: clicking tab headers switches tabs; clicking a plus area creates a new tab; mouse wheel scrolls output.

Key functions: main() X11 init, redraw().

Design:

- Used `XOpenDisplay()`, `XCreateSimpleWindow()`, and `XMapWindow()` to create the main GUI window.
- Used an event loop with `XNextEvent()` to capture `KeyPress` and `Expose` events.
- Text is drawn using `XDrawString()` on the window.
- Maintained a **text buffer** that stores all current terminal text.
- Multiple tabs are implemented by maintaining multiple buffers, each linked to a separate child process created with `fork()` and `execvp()`.
- Used pipes to redirect each child's input/output to the GUI.

---

2. Running external commands (fork + exec)

Goal: Execute arbitrary commands typed by user, including external binaries and shell invocations.

Implementation highlights:

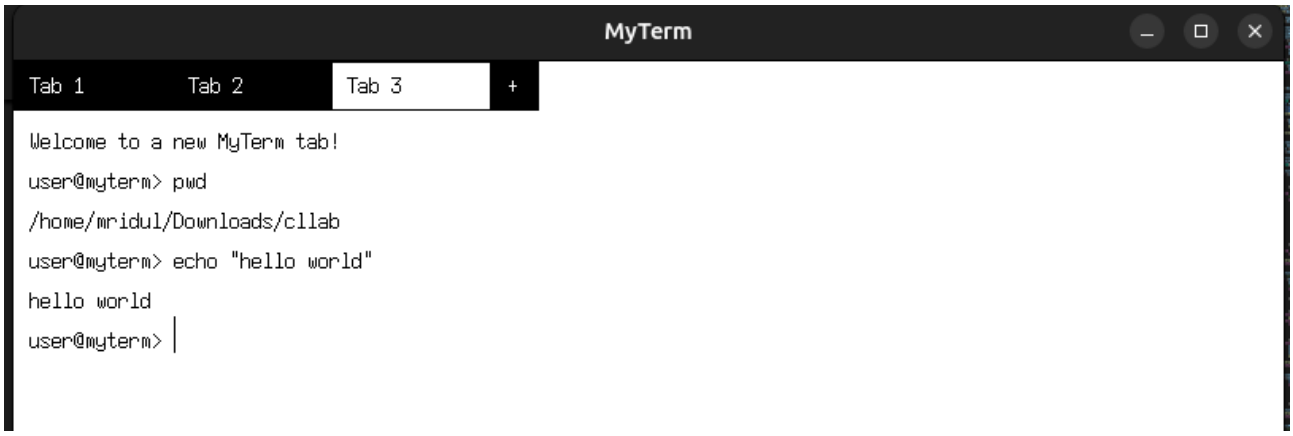
- The shell parses a command line into pipeline segments, then for each stage forks a child and execvp() the program.
- setpgid() is used to assign a process group id (pgid) to pipeline processes so we can signal them as a group.
- Standard file descriptors are manipulated in child processes using dup2() to wire up pipes and redirection.

Syscalls/libraries used: fork(), execvp(), setpgid(), dup2(), pipe(), waitpid().

Design:

- Used `fork()` to create a child process.
- In the child, used `execvp()` to execute the command.
- Parent waits for the child to finish using `waitpid()` (unless background execution is requested).
- Output of the command is captured through pipes and displayed on the X11 window.

Some examples shown below:-



---

3. Multiline & Unicode input

Goal: Accept multi-line input (line continuation using trailing backslash) and wide characters.

Implementation highlights:

- multiLineCommand accumulates continuation lines when a command ends with a backslash.
- setlocale(LC_ALL, '') and XIM/XIC are used so Xutf8LookupString() returns proper UTF-8 text.
- processEscapes() supports standard escapes like \n and \t when parsing quoted arguments.

Design:

- Used `setlocale(LC_ALL, "")` to enable UTF-8 display.
- Input captured via `read()` from X11 KeyPress events.
- Stored multiline input in buffer; rendered using Xlib functions with correct line wrapping.

---

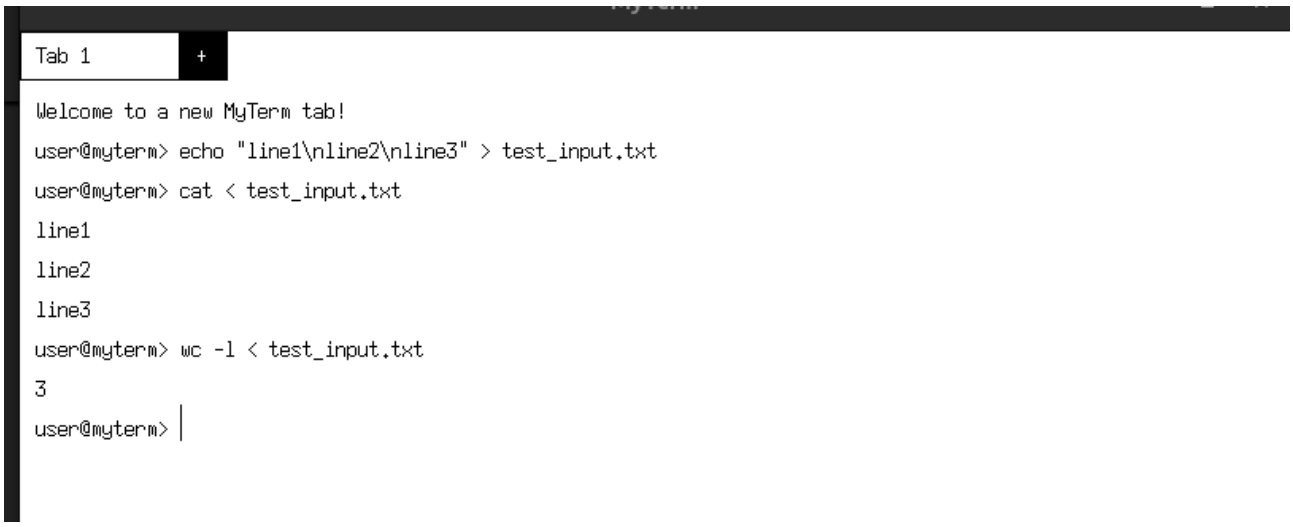
4. Input/Output redirection (<, >, >>)

Goal: Support redirecting stdin/stdout/stderr using <, >, and >>.

Implementation highlights:

- The child process inspects parsed arguments and detects <, >, >> tokens.
- Uses open() with appropriate flags and dup2() to replace STDIN/STDOUT/STDERR.
- **Design:**
    - Detected `<` symbol while parsing the command.
    - Used `open()` to open the input file.
    - Used `dup2(fd, STDIN_FILENO)` in child process before executing `execvp()`.
    - Closed the file descriptor after redirection.
- **Design:**
    - Detected `>` symbol while parsing the command.
    - Used `open()` with flags `O_WRONLY | O_CREAT | O_TRUNC`.
    - Used `dup2(fd, STDOUT_FILENO)` to redirect standard output to the file.
    - Supported combined input/output redirection like `< infile > outfile`.

A demo example shown below :-



---

5. Pipe support (|) and pipelines

Goal: Support N-stage pipelines, connecting stdout of stage i to stdin of stage i+1.

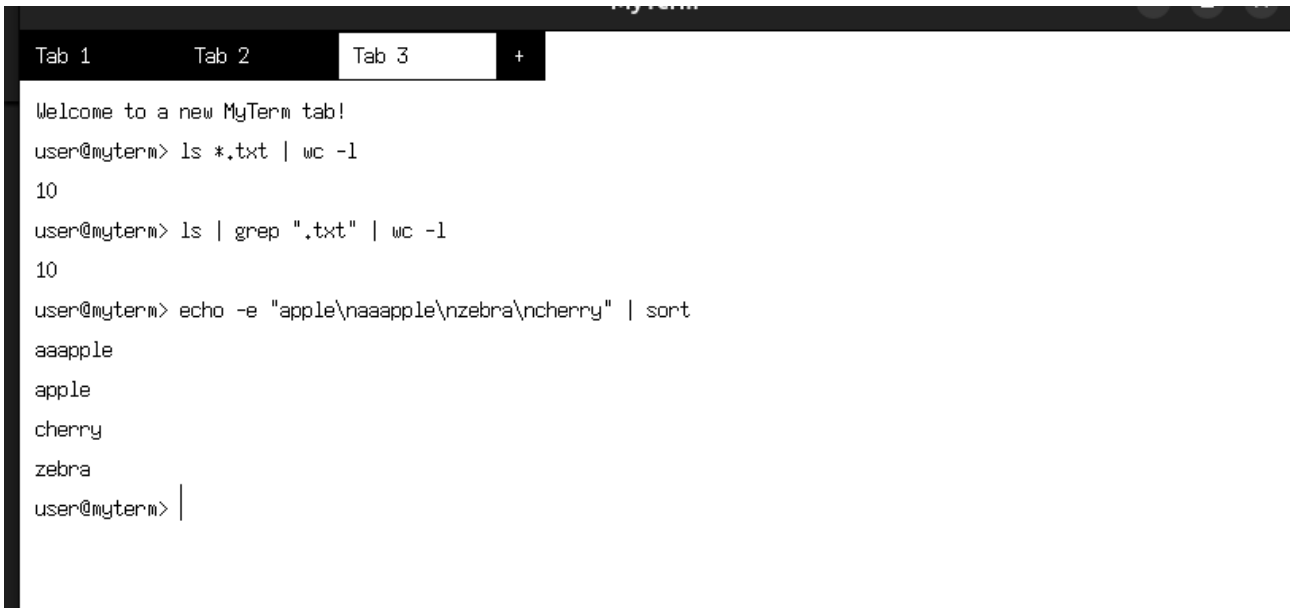
Implementation highlights:

- Shell splits input on '|' into commands and creates N-1 pipes.
- Each child receives stdin via dup2(input_fd, STDIN_FILENO) and writes stdout to the pipe write end.
- Parent sets setpgid() for each child so the entire pipeline shares a pgid.

Design:

- Parsed commands split by `|`.
- Created `N-1` pipes for `N` commands.
- For each command:
    - Created a child using `fork()`.
    - Redirected output of one process to input of next using `dup2()`.
    - Closed unused pipe ends.
- Used `execvp()` to execute each command in sequenc

An example shown below :



---

6. multiWatch command

Goal: Run multiple commands in parallel, capture their outputs, and display them with timestamps and command labels.

Implementation highlights:

- parseMultiWatch() extracts the comma-separated quoted commands inside brackets.
- For each command, parent creates a pipe and forks a child that execs 'sh -c ' with stdout/stderr redirected to the pipe.
- Parent uses select() across all pipes and the X11 connection. When data is ready, it reads, timestamps, and appends formatted blocks to the tab's text buffer.
- Ctrl+C sets a global interrupt flag so the loop terminates and children are killed.

- **Design:**
  - Created multiple child processes (one per command).
  - Each child redirects output to a temporary file (`.temp.<PID>.txt`).
  - Parent process monitors all files using `select()` or `poll()`.
  - When output is available, reads it and prints formatted output:

    ```
    "cmd", timestamp:
    -------------------
    <output>
    -------------------
    ```

  - On receiving `SIGINT` (Ctrl+C), all child processes are terminated and temp files deleted.


MyTerm
Tab 1
+
Welcome to a new MyTerm tab!
user@myterm> ls -la
total 836
drwxrwxr-x 9 mridul mridul   4096 Oct 25 09:30 .
drwxr-xr-x 7 mridul mridul   4096 Oct 24 23:36 ..
drwxrwxr-x 3 mridul mridul   4096 Oct 7 21:53 abc
-rw-rw-r-- 1 mridul mridul   327 Oct 7 21:29 abc.c
-rw-rw-r-- 1 mridul mridul   0 Oct 24 15:41 abcd.txt
-rw-rw-r-- 1 mridul mridul   0 Oct 24 15:41 abc.txt
drwxrwxr-x 2 mridul mridul   4096 Oct 24 15:41 backup
drwxrwxr-x 2 mridul mridul   4096 Oct 9 21:48 backup_folder
-rw-r--r-- 1 mridul mridul   33 Oct 24 19:39 c_files.txt
-rw-rw-r-- 1 mridul mridul 40080 Oct 10 15:58 check.cpp
drwxrwxr-x 2 mridul mridul   4096 Oct 24 15:41 config
-rw-rw-r-- 1 mridul mridul   0 Oct 24 15:41 def.txt
drwxrwxr-x 2 mridul mridul   4096 Oct 24 15:41 docs
-rw-rw-r-- 1 mridul mridul 46049 Oct 24 15:35 final2.cpp
-rw-rw-r-- 1 mridul mridul   5282 Oct 23 21:47 final.cpp

---

7. Line navigation (Ctrl+A / Ctrl+E) and key handling

Goal: Provide basic line-editing like Bash.

Implementation highlights:

- Key events are mapped to actions using KeySym and modifier state. Ctrl+A moves cursor to start; Ctrl+E to end; left/right arrows move cursor; backspace deletes.
- The prompt and cursor are redrawn per keystroke.

- **Design:**
    - Enabled terminal raw mode using `termios`.
    - Captured special ASCII values:
        - Ctrl+A → 0x01
        - Ctrl+E → 0x05
    - Maintained a cursor index in input buffer.
    - Updated cursor position and redrew line when these keys were pressed.

---

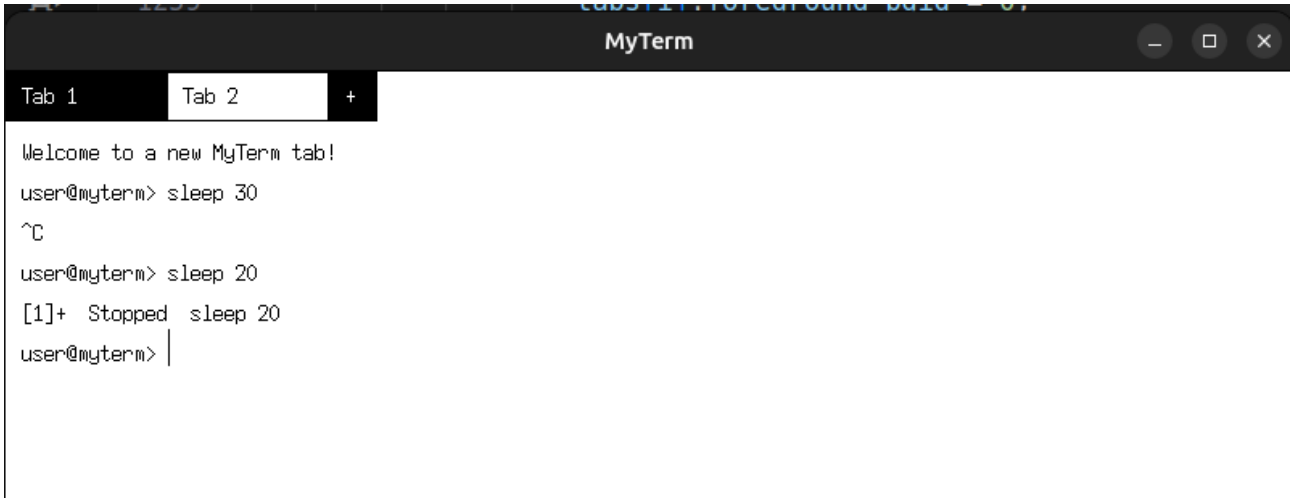
8. Interrupts and job control (Ctrl+C / Ctrl+Z, fg, bg, jobs)

Goal: Support interrupting foreground jobs, suspending them, listing jobs, and resuming via fg/bg.

Implementation highlights:

- The main shell ignores SIGINT and SIGTSTP so it is resilient to user keypresses. Foreground pgids receive signals via kill(-pgid, SIGINT) or SIGTSTP.
- jobs vector stores <pgid, command>. fg uses kill(-pgid, SIGCONT) and waits with waitpid(-pgid, WUNTRACED).
- SIGCHLD handler sets a flag g_sigchld_received; the main loop reaps zombies using waitpid(..., WNOHANG) and updates job state.
- **Design:**
    - Used `signal(SIGINT, handler)` and `signal(SIGTSTP, handler)`.
    - On `Ctrl+C`, child process is killed using `kill(pid, SIGINT)` but shell remains alive.
    - On `Ctrl+Z`, moved process to background by not calling `waitpid()`.

- Maintained a list of background jobs.



---

9. Searchable shell history (Ctrl+R)

Goal: Maintain last 10,000 commands, support history to show last 1000, and reverse-search.

Implementation highlights:

- history vector is capped at 10,000; history command prints last 1000 entries.
- Ctrl+R enters search mode; performSearch() finds the most recent command containing the search term. finalizeSearch() implements the selection logic: exact match preferred, otherwise most recent with longest common substring > 2.

Design:

- Stored up to 10,000 commands in a vector or file (`.myterm_history`).
- Implemented `history` command to show last 1,000.
- On `Ctrl+R`, prompted “Enter search term”.
- If exact match found → printed latest matching command.
- Else → computed longest substring matches (length > 2).
- Else → printed “No match for search term in history”.

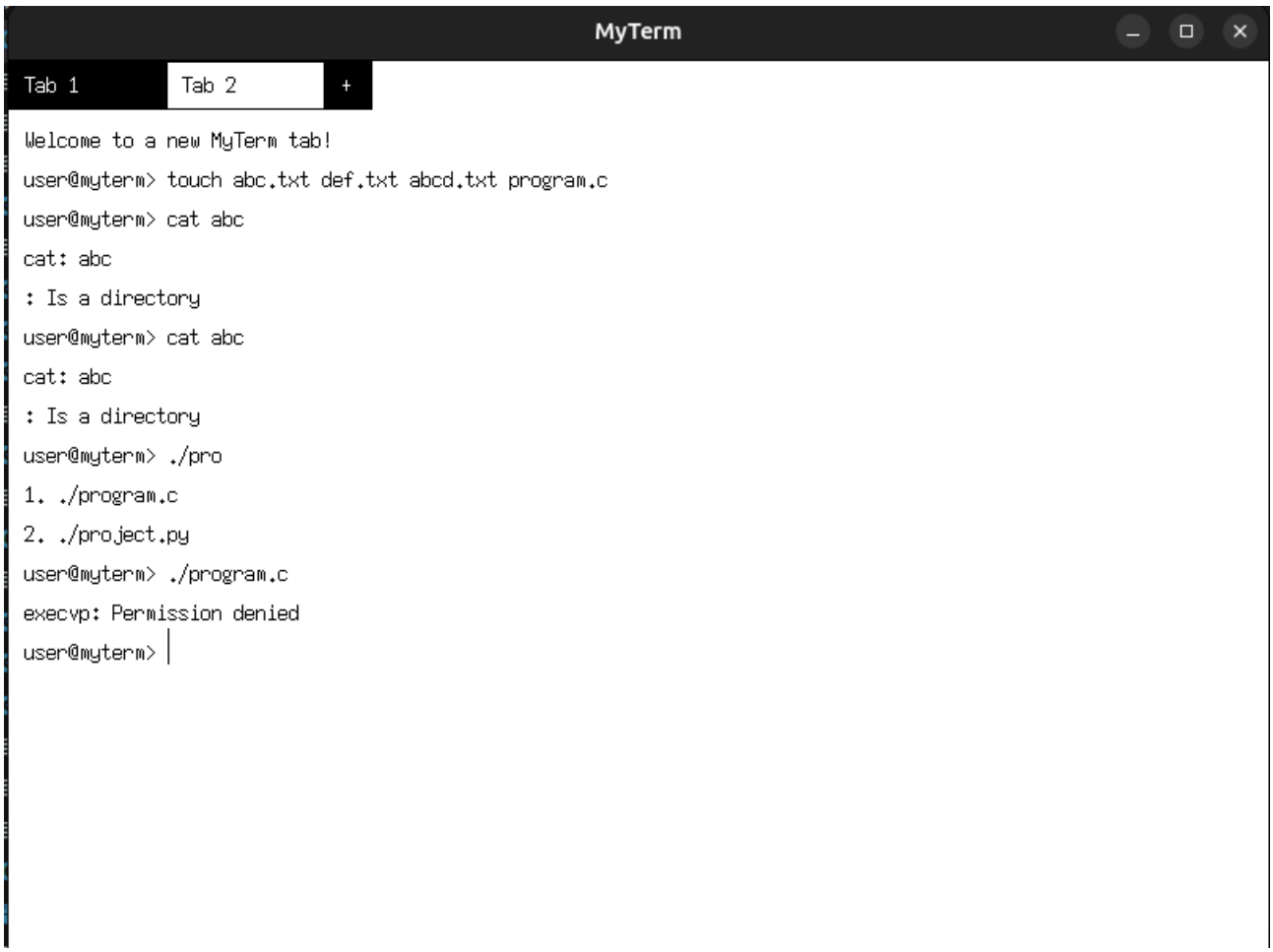
---

10. Auto-complete (Tab)

Goal: File-name completion in current working directory.

Implementation highlights:

- On Tab press, get word before cursor and call glob() with pattern*. If one match, replace; if multiple, compute longest common prefix; if ambiguous, list choices and accept a numeric selection.
- **Design:**
    - Captured Tab key press event (ASCII 9).
    - Used `opendir()` and `readdir()` to get filenames in current directory.
    - Matched prefix with user input.
    - If one match → completed automatically.
    - If multiple matches → printed numbered options and asked for user selection.
    - If none → did nothing.



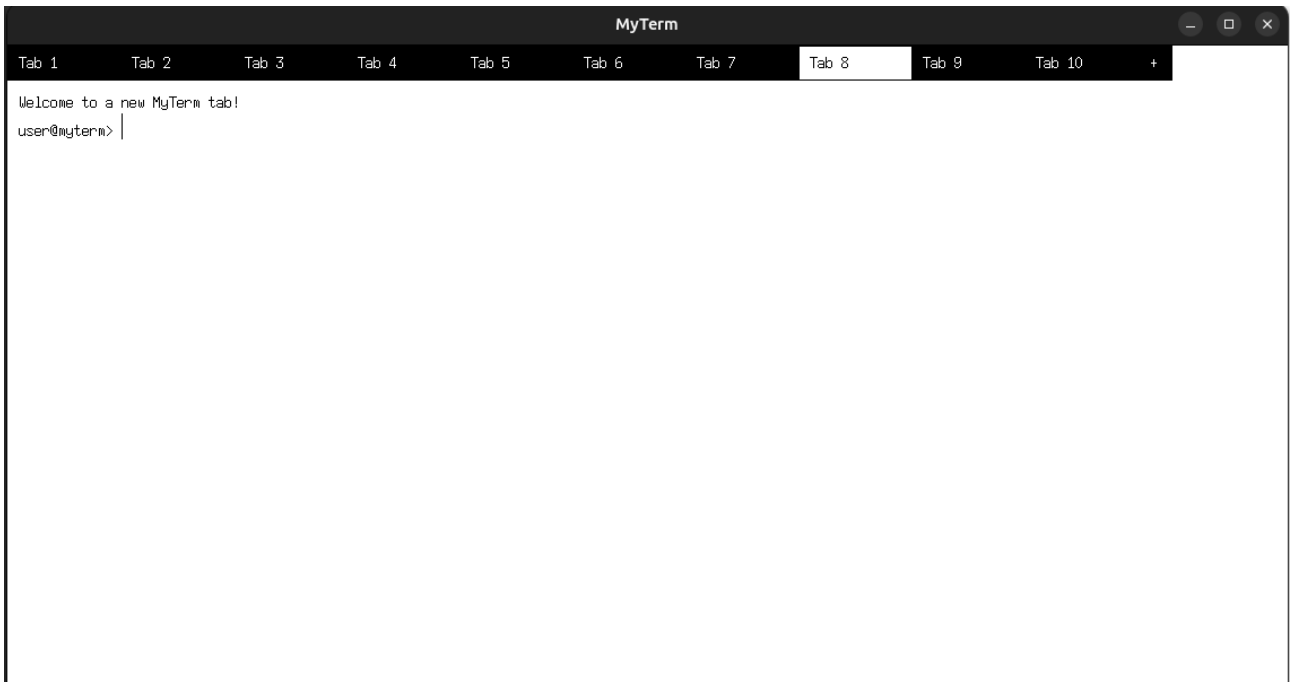
---

11. Multiple tabs

Goal: Each tab behaves as an independent shell instance with its own buffer and state.

Implementation highlights:

- tabs is a vector of ShellTab structs. Clicking on tab area changes activeTabIndex. New tabs are created via a plus button or Ctrl+N.



Additional Notes

- **Languages Used:** C++ (with Xlib and POSIX syscalls)
- **Libraries:** `<X11/Xlib.h>`, `<unistd.h>`, `<fcntl.h>`, `<sys/wait.h>`, `<signal.h>`, `<termios.h>`
- **Error Handling:** Used proper `errno` checks for system calls.
- **Memory Management:** Used dynamic buffers for flexible command input.
- **Testing:** Each feature verified using real shell commands (`ls`, `wc`, `cat`, `gcc`, etc.)

Some of the important syscalls used in this project :-

1. `dup()` / `dup2()`

- **Purpose:** Duplicate a file descriptor.
- **Usage:** Redirect `stdin` or `stdout` (e.g., for < or > redirection).
- **Difference:**
  - `dup(oldfd)` → returns a new descriptor.
  - `dup2(oldfd, newfd)` → forces it into `newfd` (useful for redirection).

Example:

```
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO);   // Redirect stdout to output.txt
printf("Hello MyTerm!\n"); // Writes to file, not screen
```

```
close(fd);
```

---

2. `execlp()` / `execvp()`

- **Purpose:** Replace the current process with a new program.
- **Usage:** Used in the **child process after fork()** to run a command.
- **Difference:**
  - `execlp("ls", "ls", "-l", NULL);` → List of arguments.
  - `execvp("ls", args);` → Array of arguments.

Example:

```
char *args[] = {"ls", "-l", NULL};
execvp(args[0], args); // replaces current process with `ls -l`
```

---

3. `fork()`

- **Purpose:** Create a new process (child).
- **Returns:**
  - `0` in the child.
  - Child's PID in the parent.

Example:

```
pid_t pid = fork();
if (pid == 0) {
    printf("Child process\n");
} else {
    printf("Parent process, child PID = %d\n", pid);
}
```

---

4. `signal()`

- **Purpose:** Handle or ignore signals like `SIGINT` (Ctrl+C) or `SIGTSTP` (Ctrl+Z).
- **Usage:** Helps MyTerm handle interruptions without quitting.

Example:

```
#include <signal.h>
void handler(int sig) { printf("Caught signal %d\n", sig); }

int main() {
    signal(SIGINT, handler); // Handle Ctrl+C
    while (1) pause();       // Wait for signals
}
```

---

5. `pipe()`

- **Purpose:** Create a communication channel between processes.
- **Usage:** Connect `stdout` of one command to `stdin` of another (`|`).

Example:

```
int fd[2];
pipe(fd);
if (fork() == 0) {
    dup2(fd[1], STDOUT_FILENO); // child writes
    close(fd[0]);
    execlp("ls", "ls", NULL);
} else {
    dup2(fd[0], STDIN_FILENO);  // parent reads
    close(fd[1]);
    execlp("wc", "wc", "-l", NULL);
}
```

---

6. `select()`

- **Purpose:** Monitor multiple file descriptors (wait until one is ready for I/O).
- **Usage:** Used in `multiWatch` to read from multiple child outputs efficiently.

Example:

```
fd_set fds;
FD_ZERO(&fds);
FD_SET(fd1, &fds);
FD_SET(fd2, &fds);
select(max(fd1, fd2) + 1, &fds, NULL, NULL, NULL);
```

---

7. `poll()`

- **Purpose:** Similar to `select()`, but better for large sets of FDs.
- **Usage:** Alternate way to monitor multiple I/O sources.

Example:

```
struct pollfd fds[2];
fds[0].fd = fd1; fds[0].events = POLLIN;
fds[1].fd = fd2; fds[1].events = POLLIN;
poll(fds, 2, -1);
if (fds[0].revents & POLLIN) read(fd1, buf, 100);
```

---

8. `open()`, `read()`, `write()`

- **Purpose:** Perform file I/O without stdio (`fopen`, etc.).
- **Usage:** Used for redirection, reading input/output manually.

Example:

```
int fd = open("file.txt", O_RDONLY);
char buf[100];
int n = read(fd, buf, sizeof(buf));
write(STDOUT_FILENO, buf, n); // Print to terminal
close(fd);
```

---

9. `chmod()`

- **Purpose:** Change file permissions.
- **Example:**

```
chmod("file.txt", 0644); // rw-r--r--
```